Online Game Design
2025-2026
Technical Design Document



Daniel Bonardi
Marco Caramazza
Eduard Ionut Zugravu

TDD Revision 4c. May 2024

Meet Team Tri-Side Die



Daniel Bonardi

Lead Programmer, Game Designer, Game programmer

daniel.bonardi@mail.polimi.it

302564 (Polimi)

Marco Caramazza

Lead Artist, Game Designer, Game Programmer

marco.caramazza@mail.polimi.it

270885 (Polimi)





Eduard Ionut Zugravu

Game Designer, Audio Artist, Art Artist

eduardionut.zugravu@studenti.unimi.it

83970A (Statale)

Changelog....4
Changelog....4
1 Project Goal....5
2 Innovation....6
3 Provided Services (Beside the Game)....7
4 Client side....8
5 Workload Estimation....9
5.1 Similar games workload analysis....9
5.1.1 Stardew Valley....9
5.1.2 Terraria....9
5.1.3 Core Keeper....10
5.1.4 Apico....10
5.2 Estimation....11
6 General Architecture....13
6.1 Frontend....13
6.1.1 Hardware....13
6.1.2 Software....13
6.1.3 Services....13
6.2 Backend....13
6.2.1 Hardware....13
6.2.2 Software....13
6.2.3 Services....14
6.3 Workload Capacity....14
7 External Services....14
8 Connection....14
8.1 Global Infrastructure....14
8.2 Network Requirements....15
8.3 Connection flow....16
8.4 Session joining process....16
8.5 Client-host communication....17
9 Delivery....19
9.1 Estimated Delivery Time....19
9.2 Delivery Platform....19
9.3 Gantt Chart....20
10 Cost Estimation....21
11 References....23

Changelog

Creation Date: 10/05/2026
Last modification: 01/07/2026

Changelog		
When	What	Who
30/03/2026	Client side section	Daniel Bonardi
01/04/2026	Project goal section	Daniel Bonardi
01/04/2026	Innovation section	Daniel Bonardi
10/04/2026	Provided services section	Daniel Bonardi
10/04/2026	Workload estimation section	Daniel Bonardi
10/04/2026	General architecture section	Daniel Bonardi
10/04/2026	External services section	Daniel Bonardi
11/04/2026	Connection section	Daniel Bonardi
12/04/2026	Delivery section	Daniel Bonardi
12/04/2026	Cost estimation section	Daniel Bonardi
12/04/2026	References section	Daniel Bonardi
13/04/2026	Refined workload estimation	Daniel Bonardi
14/04/2026	Added more images	Daniel Bonardi
14/04/2026	Reevaluated costs	Daniel Bonardi
14/04/2026	Reevaluated costs	Marco Caramazza
15/04/2026	Added sequence diagrams	Daniel Bonardi
15/04/2026	Added cover page	Marco Caramazza
15/04/2026	Added summary page	Daniel Bonardi
16/04/2026	Team composition	Marco Caramazza
17/04/2026	Fixed system requirements	Daniel Bonardi
17/04/2026	Fixed GDD references	Daniel Bonardi
01/07/2026	Re-evaluated cost estimation	danielcadorna.17@gmail....

1 Project Goal

In this project we aim at providing a user experience based on a stable and low-latency co-op environment, to allow players to seamlessly change from single player to co-op mode. From a technical standpoint, we opted for a Listen Server architecture that allows up to 4 players in a single session with a 24/7 availability thanks to Unity Gaming Services.

Following Unity's Netcode for GameObjects official documentation, our implementation utilizes the host's device both as a client and as the authoritative server. According to the documentation, this approach offers significant advantages, mainly in terms of contained infrastructure costs and simplified Relay setup, but it is also subject to several disadvantages that need to be mentioned and addressed to ensure they do not compromise the user experience we envisioned for our players.

- **Host performance and Workload**:
  According to Unity documentation, *"the network performance relies on the residential internet connection of the host and their machine's output efficiency"*.
  While the host's device needs to carry a more intensive workload, due to its own graphical rendering and the computation of other players actions, the game's 2D pixel art style significantly reduces GPU overhead. This allows the CPU to prioritize other computationally heavy operations, such as the bees artificial life simulation and the network data synchronisation, without harmfully impacting the host performance.

- **Latency disparity**:
According to Unity documentation, *"The host player may have a large latency advantage over the remote players"*.
Since the game focuses on asynchronous simulation and collaborative hive management rather than competitive mechanics causing conflicts among players, any kind of latency advantage granted to the host is negligible and is highly unlikely to impact in a harmful way the cooperative gameplay loop.

- **Security and Anti-Cheat**:
  According to Unity documentation, *"With access to all of the game world's information, the host player has an easier opportunity to cheat"*.
  In a collaborative environment designed for friends, the incentive for cheating is low. Since the host acts as the "Ultimate source of truth", we think it is acceptable for the host to cheat in their own session, as it doesn't affect the global integrity of the other players' individual worlds.

- **Session persistence and Disconnection**:
  According to Unity documentation, *"The server, i.e. the game ceases to exist when the host player leaves the game"*.
  To mitigate the loss of progress made during the online session when the host leaves we are implementing a periodic session snapshot, this allows players to update their hive data whenever a predetermined number of changes are made. When the host leaves the session either voluntarily or by accident (session crash), the system sends the individual players back to their own hive in single player mode, with minimal amount of data lost.

As noted in the official Unity documentation, a *"Listen server architecture is a popular choice for single player games that want to provide the option to add a friend into an existing game world. Listen servers are best suited for a smaller player group (< 12) and games that don't require a persistent world."*
This perfectly aligns with our vision of the project.

2 Innovation

In this section we will explain how, even though the core technology required for our project relies on already existing frameworks, it still requires some innovation on how data synchronization is handled for decentralized simulations.
To solve the various problems we thought about implementing these solutions:

- **Game mode switching**:
  Unlike traditional multiplayer games where players join a static level (lobby and game scenes), our game needs to switch between single player mode and co-op mode as smoothly as possible. For this reason the players need to be connected to NGO and be inside a lobby as soon as they start the game in single player mode. This allows us to hide to the players the step of creating a lobby. This creates a new problem because when a player sends a request to another player to join their room, they need to leave the one they are currently in and enter the other player's one. At this point there are two cases:
  - The host accepts the connection: This is the simple case, with no problem at all. Since the player is already inside the lobby, it's just a matter of merging and syncing all the player's hives in the room.
  - The host rejects the connection: In this case the requesting client is kicked from the lobby and he needs to join a new lobby to allow other players to join him. All of this needs to happen without any change in the scene and with just a notification to the rejected client, hiding to him all the complexity behind the whole process.

- **Cooperative Grid-based data syncing**:
  Our game features a hive editing system that is divided in three sub-modes: cell editing mode, furniture editing mode and workbenches editing mode.
  The most notable one is the cell editing mode, where players can mark cells as pending additions and pending deletions if they satisfy certain conditions. Pending additions and deletions must be broadcasted only to players that are in edit mode, while still passing through the host to verify the correctness of the operation. Every single cell is checked by the host before being marked as a pending addition or deletion, this way we can easily avoid the problem of two players trying to interact with the same cell at the same time. Furthermore, to decrease the volume of data being sent through the network, the clients send packets containing only the changes being made instead of the whole hive state, allowing for a faster data transfer and ensuring that building feels instantaneous also for the clients.
  More detailed information about the edit mode features can be found in the Game Design Document section 5.2.

- **Shared entity authority**:
  While in co-op mode, basically every resource becomes shared for every player. Although in our architecture the host is the "Ultimate source of truth", authority over specific bees or facilities can be temporarily passed between players to ensure a low latency experience.

3 Provided Services (Beside the Game)

In this section, we outline the supporting infrastructure required to maintain the envisioned environment and user experience outside of the main gameplay loop.

- **Room discovery**:
  To streamline the co-op feature and make it easier for players to join another player's session, we chose to go for a hybrid approach utilizing both Steamworks API and Unity Lobby. This hybrid approach allows players to join sessions based on social context and needs:
  - **Join through Steam friends**:
    Thanks to Steam's API we can use the already present lobby system, and allow players to see active sessions directly in their friend list and join without manually entering a room code.
    This join method is specifically thought for people that want to play with their friend, using Steam's API to hide to the user part of the complexity of joining a room.
  - **Join by room code**:
    This join method instead is thought for people that are not on a friend list. The host generates a unique alphanumeric code through Unity Lobby, when a player wants to join a session, he simply needs to enter the code and send a join request.This method might be useful to allow people that were just met (for example through a community channel), without too much effort.

- **Unity Relay**:
  Since we opted for a Listen Server architecture, we provide connectivity through Unity Relay service. Unlike traditional peer-to-peer methods that require NAT punch-through or manual port forwarding, Unity Relay acts as a secure bridge to establish a connection between two players. This ensures that players can connect to a host even if they are behind strict firewalls by routing the traffic through a dedicated relay node, hiding all the connection complexity from the user.

- **Beta feedback and bug reporting**:
  Given the fact that the project is intended for an open beta, we thought about implementing a dedicated diagnostic toolkit. Telemetry and log files along with user-submitted feedback, will be transmitted to our backend, using Unity Cloud Diagnostics, to help developers identify and fix edge cases and unexpected behaviours inside the game code.

- **Persistent user profiles**:
  While the game logic is handled by the host, all player data such as hive data, unlocked and placed furniture, etc., needs to be synced in a lightweight save service in the cloud, such as Unity Cloud Save, to ensure that progress is retained across different play sessions, both in single and co-op mode. This service provides a centralized database that allows storing player specific JSON data. In this way we can minimize the amount of data being saved locally in a player's device, and also reduce the risk of cheating, making it harder for players to access and modify their own data.

- **Community communication channels**:
  Given our target audience of "socializer" players, we will provide a multi-purpose Discord server that serves as a primary hub both for community building and developer pipeline. In this way players not only will be able to find co-op partners and share achievements and hive strategies, but also they could directly contact the developers team regarding in-game bugs or receive real-time announcements regarding updates and server

maintenance. Furthermore, since the game already implements a chat system for players inside the same session but not a voice chat, thanks to discord voice channels it would be easier for players that just met to communicate.

4 Client side

Minimum requirements:

CPU	Intel Pentium G4560 AMD Athlon 3000G
RAM	4 GB
GPU	Intel HD 4000 AMD Radeon R5 230 or equivalent integrated graphics
Storage	2 GB
OS	Windows 10/11 - 64 bit
DirectX	Version 11
Target resolution	720p @ 30fps
Connection	Download: 5 - 10 Mbps Upload: 3 - 6 Mbps
Additional notes	Input system with mouse and keyboard only

Recommended requirements:

CPU	Intel Core i3-10100 AMD Ryzen 3 3100
RAM	8 GB
GPU	Nvidia GTX 1050 AMD RX 560 or equivalent integrated card
Storage	3 GB
OS	Windows 10/11 - 64 bit
DirectX	Version 11
Target resolution	1080p @ 60fps
Connection	Download: 15 - 25 Mbps Upload: 10 - 15 Mbps
Additional notes	Input system with mouse and keyboard only

5 Workload Estimation

In this section we analyze the workload of other famous games published on steam and estimate the workload for our game.

5.1 Similar games workload analysis

Before actually estimating the workload of our game, we prefer taking reference from the workload of other similar games.

5.1.1 Stardew Valley

Stardew Valley is a farming life simulator released in February of 2016 and published by ConcernedApe. We chose this game for comparison because it's the benchmark for modern 2D simulation games and features a 4-player co-op mode where players manage a shared environment. For Hivemind it represents the best case scenario for community growth.



5.1.2 Terraria

Terraria is an open world sandbox survival game released in May of 2011 and published by Re-Logic. We chose this game for the comparison because similarly to Stardew Valley it features an online co-op feature with a 2D pixel art style, with an even bigger community.



5.1.3 Core Keeper

Core Keeper is an open world survival game released in August of 2024 and published by Fireshine Games. We chose this game for comparison because the game uses a grid-based building system similar to our hive editing mechanics. It also allows up to 8 players in the same session, placing higher demand on the host's CPU for world synchronization.



5.1.4 Apico

Apico is a specialized beekeeping simulator released in May of 2022 and published by Whitethorn Games. We chose this game for comparison because it features a 4-player online co-op feature with 2D pixel art style. This game provides the most accurate technical comparison for the specific niche of player we are targeting even though the game maintains a smaller community than the one expected for our game.

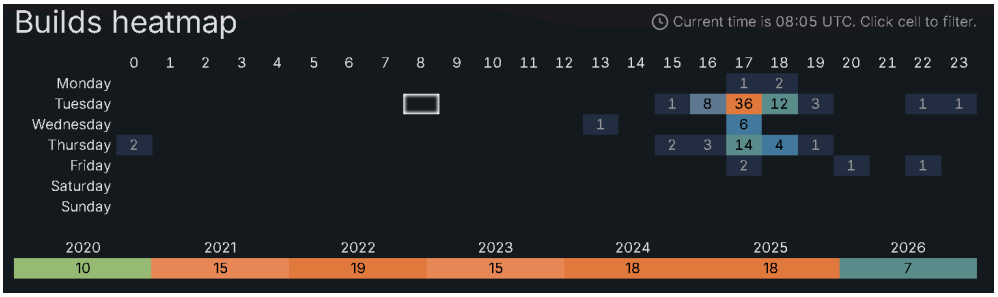


5.2 Estimation

Based on the lifecycle of other similar indie games like Stardew Valley and Core Keeper, we expect the traditional “Peak and Decay” trajectory for the usage graph.
Let’s analyze in detail the different phases of our game lifecycle:

- **Peak on Launch**:
  It is more than likely that we will reach our all-time peak within the first two weeks from launch, usually it happens around the end of the first week. To slightly mitigate the problem of having too many concurrent players at the same time of the day and avoid wasting resources we can adopt some non-technical solutions:

  - **Launch day**:
    We could set the launch day on Monday or Tuesday, with the purpose of allowing students to start playing immediately after launch, while working people will most likely wait until the weekend to start playing. This allows us to have a better distribution of players among the first weeks.

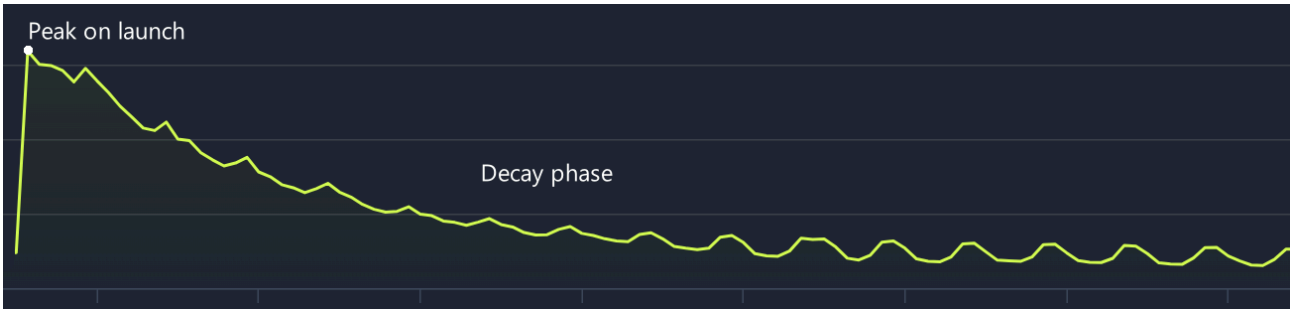


    This screenshot was taken from steamdb.info and it shows the days of the week when the game “The Sims 4” released its updates. And in particular, even though it is not shown in the photo, the game was released on Tuesday, September 2, 2014, making us think that they might have applied the same reasoning as our launch day strategy.

  - **Off-peak incentives**:
    We could implement a system that gives more rewards based on the time of the day a player is active. This kind of system is mostly used in MMORPGs like Elder Scrolls Online or Dark Age of Camelot to alleviate the problem of too many concurrent players. Since our game has a wide variety of bees, we could specify for each type of bee a specific time window during the day, during which their statistics are slightly buffed.
    Another interesting system used to incentivize the distribution of playtime is the “Happy Hour” system, which usually is a timed window that provides different types of bonuses. In our game we could have timed events that happen at a specific time during the day, but that happen at different times depending on the player's geographical area, during which players could earn extra rewards for expeditions, or all bees receive a buff in their statistics.

- **Decay phase**:
  Following the graph on player retention, within the third month the playerbase should be reduced down to the 40% of the peak. This problem can be already mitigated with the techniques explained in the previous point, allowing to delay the 40% mark to around the sixth month, and after the first year the new curve quickly synchronizes with the original decay. To overcome this problem and extend the lifecycle of our game we can deliver

constant updates and new content through DLCs.
DLCs are the most important aspect in this phase because they can avoid losing players but also attract new or returning players. In our game we already have a good amount of characters and enemies (bees and other insects), so we could decide to implement a limited number of them for the first release and add more with future DLCs. Usually the timing for the first 2 DLCs is around the ninth month and a couple of months after the first year, but it is always important to keep track of the number of active players to fine tune the envisioned model and make more precise decisions on the release date of the DLCs.



Here is a summed up table with the estimated workload.

Metric	Target (30 days)	Notes
Total registered users	5.000	This number is based on the “Socializer” player interest in the A-Life genre and relative to indie games
Peak concurrent users	500	We assume a 10% is a peak-to-total suitable ratio, which is pretty common among indie games betas
Avg. session duration	2 hours	This feels like a good estimation, since we need to consider the fact that the game can be played both in single player and co-op mode. Players usually spend more time playing with friends. Also, the game is targeting hardcore players, meaning that it incentivises longer play sessions.
Avg. entities to manage	~200 AI agents	Based on the speed a player can create new bees and the fact that a session can manage up to 4 players

Our infrastructure modelled with Unity Relay and Steam Lobbies is supposed to handle initial 500 concurrent users without additional dedicated hardware costs.
Initially we plan on using Unity Gaming Services free tier, where Unity Relay allows up to 50 concurrent sessions. Once this limit is exceeded Unity Gaming Services automatically transitions to the “Pay as you go” tier, charging $0.16 per additional average CCU.

6 General Architecture

In this section we will describe the technical layers of the host-authoritative model based on Unity Netcode for GameObjects.

6.1 Frontend

The frontend represents the game executable running on player machines (host included). It is responsible for the hive representation, input handling and player actions broadcasting.

6.1.1 Hardware

The intended hardware is a standard gaming PC and peripherals with a focus on mouse and keyboard since the game mechanics require all of the input to be retrieved from them.
More information about the hardware specifications can be found in section 3.3 of the game design document.

6.1.2 Software

Game Engine: Unity 6.3 LTS (6000.3.10f1)

Rendering: Universal Rendering Pipeline (URP) using 2D framework

Networking framework: Unity Netcode for GameObjects

Transport Layer: Unity Transport (UTP)

6.1.3 Services

Client side prediction: In edit mode the client shows a “ghost” object, representing where the cell or furniture is going to be placed if the operation is successful.

Interpolation and extrapolation: Used to smooth out the movement of bees in client’s hives, especially in cases where a client's connection is unstable (low connection or package jittering).

6.2 Backend

In our listen server architecture, the backend can be seen as an hybrid layer consisting of the host’s authoritative device and the supporting cloud services (Unity Gaming Services and Steamworks).

6.2.1 Hardware

Since our architecture is to be considered hybrid we must divide between host machine and third party infrastructures.
Managed cloud infrastructure is provided by Unity Gaming Services and Steamworks, which includes distributed cloud servers, global relay nodes and scalable service hosting.
As for the host machine, it is important to specify that it needs to be slightly more powerful than the client’s machine since it has a more important workload. However this aspect has been already considered in the hardware specification in section 3.3 of the game design document, since there is no limitation to who is to be considered the host, meaning every player might happen to be the host of a session once in a while.

6.2.2 Software

The software relies on Unity Netcode for GameObjects for hive state’s synchronization.
The host maintains all the authoritative logic. Any attempt of the client at creating or deleting a cell, placing furniture and every other action that involves the hive state, a ServerRPC is called, the host checks that the action is allowed, updates the corresponding data structures, so that

the next client requests are checked on the correct state. Once the action is confirmed as allowed, the host broadcasts a ClientRPC to every client so that whoever needs the new data, the correct structures are updated.

6.2.3 Services

Unity Transport: This is the underlying protocol using UDP, for high performance and reliable data transfer.

Heartbeat: Lobbies created both with Unity NGO and Steamworks API need to be kept alive through a background task as long as the session exists, meaning when the host is in game.

6.3 Workload Capacity

The infrastructure is designed to be scalable by nature, allowing for:

Decentralized processing:
Even though the host is the "Ultimate source of truth" the A-life simulations of the bees are handled by the clients themselves.

Bandwidth optimization:
We can make use of network culling, that is a technique that allows the server to reduce the amount of data being sent through the network. A player receives updates for hive cells and bees only when they are within their camera view. This way even when the scaling of the merged hive becomes huge, the network packets remain of a manageable size.

Stability:
Considering a 30 Hz tick rate for server updates, each session might end up consuming roughly 250 kbps. Our choice of Unity Relay can handle a huge amount of these concurrent data flows making it resilient to the anticipated 500 concurrent users.

7 External Services

In this section we focus on identifying third-party platforms and tools we decided to integrate in our workflow and that are not part of the cloud infrastructure.

Steamworks SDK:
This is essential for publishing the PC version of the game. It provides the API for the "Join through Steam friends" feature.

GitHub:
This is essential for tracking development progress and version control.

Discord:
This is essential for the creation of the community server as discussed in section 3 of this document.

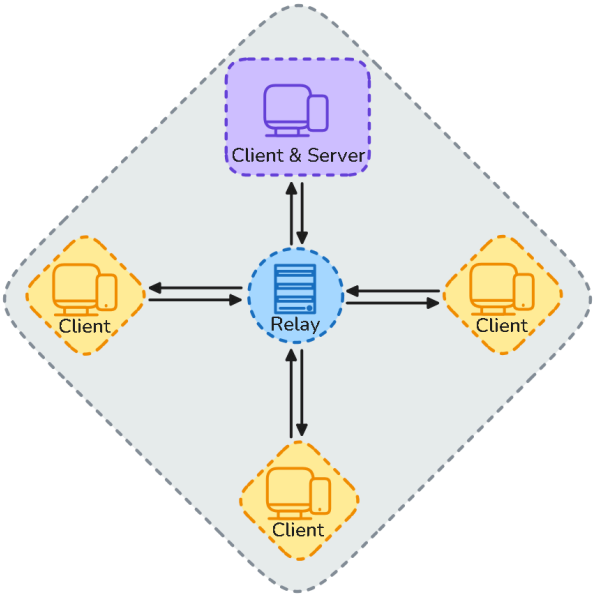
8 Connection

8.1 Global Infrastructure

Our infrastructure follows the star network topology, built around Unity Relay nodes acting as proxies. According to this model the relay server acts as a central hub, allowing every player to communicate with each other regardless of their local router's security settings.

8.2 Network Requirements

To ensure the best user experience for the artificial life simulation, we prioritize stability over raw speed.



Bandwidth requirements:
It is important to note that host and client might have different requirements given the difference in their workloads.

- Host: Requires a minimum of 500 kbps in upload since he must broadcast all the actions and modifications made by other players to the merged hive.
- Client: Requires around 250 kbps in download since they simply have to send the server their actions , but they need to mostly receive feedback about their own actions and the updates of other players.

Latency:
We aim at achieving a stable connection that always remains under 150 ms of round trip time.

Connection type:
Reliable and unreliable UDP via default Unity transport.
Since Unity transport allows for both reliable and unreliable packet delivery, we plan on using the reliable mode for the most important part of syncing the hives, like cell and furniture editing actions, while we plan on using unreliable packet delivery for data that needs frequent updates like bees position.

Quality of Service:
Here we list in order the priorities for our service:

1. **Hive integrity**: Sending updates about hive changes is among our top priority since we prioritize the artificial life simulation experience through stability. The packets for this mechanic are sent with reliable RPCs to ensure the hive layout never desyncs.

2. **Resources syncing**: Also hive resources need to be synced to ensure that players are always aware of how many resources they have available and plan accordingly.

3. **Bees position**: We use linear interpolation on the client side to smooth out the movement, allowing a lower amount of data relative to the bees position being sent through the network, and allowing us to drop old position packets if the network becomes congested.

4. **Relay selection**: We use Unity QoS Discovery Service to ping multiple regional datacenters to automatically select the one with the best performance relative to the host's relay allocation.

8.3 Connection flow

The first step in the connection flow is the **allocation** of the relay. When a player starts a session he is automatically marked as the host (since he is the only player) and during loading he sends an allocation request to the nearest Unity Datacenter.
The second step is the **binding**. This allows the player that established a connection with the relay server reserved by the allocation service during the allocation process. After this process is completed, the relay server can discover the player's IP address and exchange packets.
The last step is the **joining** process. In this phase the clients connect to the host player's allocation on the relay server. Once this step is completed, clients can exchange packets with the host, but not directly, since all packets are sent to the relay and then proxied to the host.

8.4 Session joining process

1. **Join game**: The player navigates through the UI and reaches the screen to enter one of his hives, select one and click play.

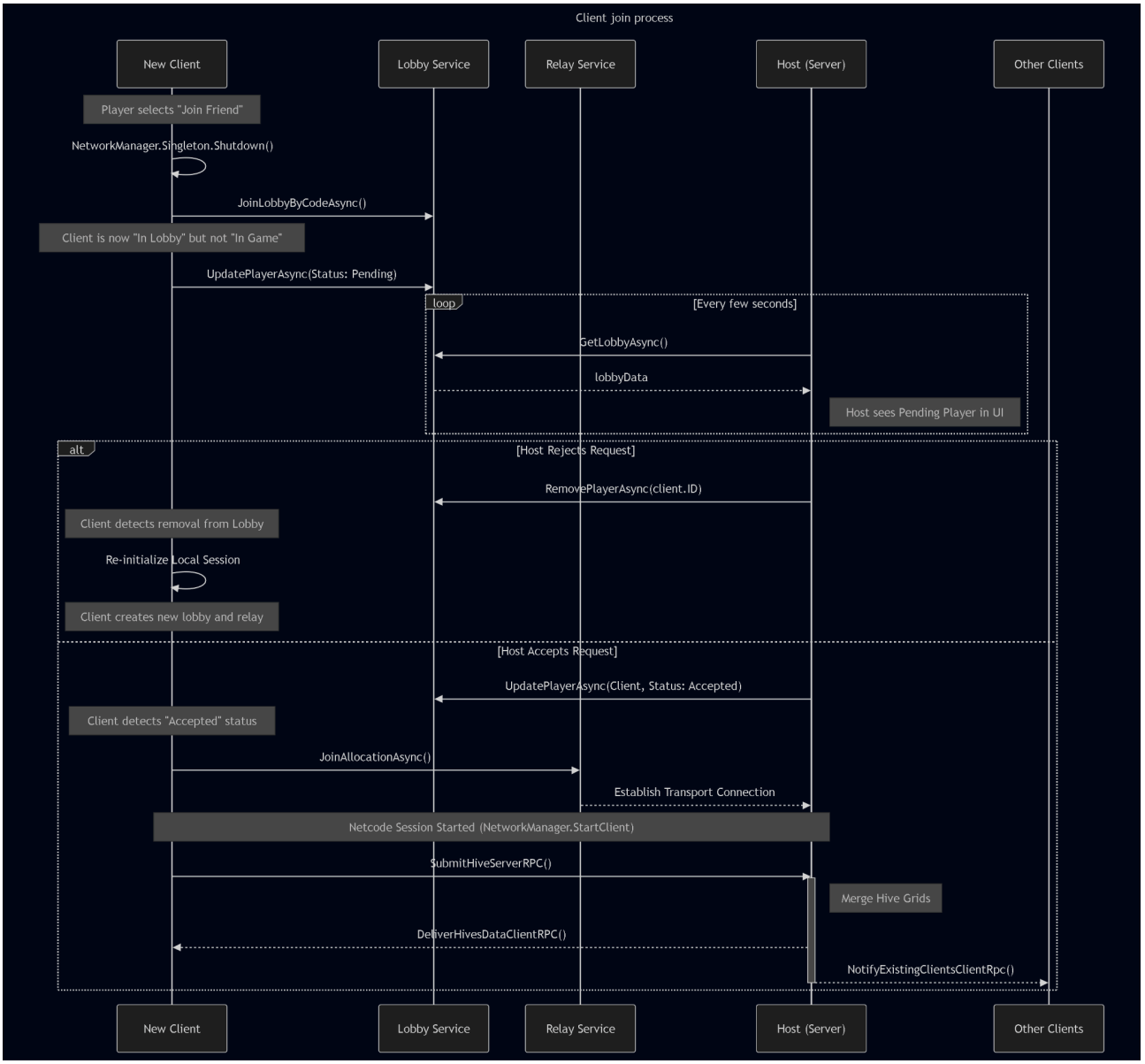
2. **Send request**: The player enters a friends room code and sends the request or uses the Steam overlay to check which one of his friends are playing the game and are in a session he can join, then sends a request. When sending a request the client needs to connect to the same session of the host, meaning he has to disconnect from his own NGO connection and temporarily join the one of the player he is sending the request to.

3. **Handle request**: The host, every 2 seconds, polls from the lobby data to check for incoming requests. When he finds a request he can decide whether to accept or reject it.
    a. **Reject**: The client sending the request is sent a notification about the rejection and then is kicked from the host session. At this point he needs to create a new session for himself.
    b. **Accept**: The client sending the request is notified about the accepted request.

4. **On request accepted**: The client sends over the network his hive data.

5. **Host computation**: The host receives the new client's hive data and computes the optimal position based on the position of the hives of all players in the session, and then broadcasts the computation to all clients.

6. **Broadcast**: The new client receives the hive data of all the players in the session and renders everything. The other clients receive the hive data of the new client and its position and then render it.



8.5 Client-host communication

For the client-host communication we use an example of how the client can request the painting of a cell inside the merged hive.

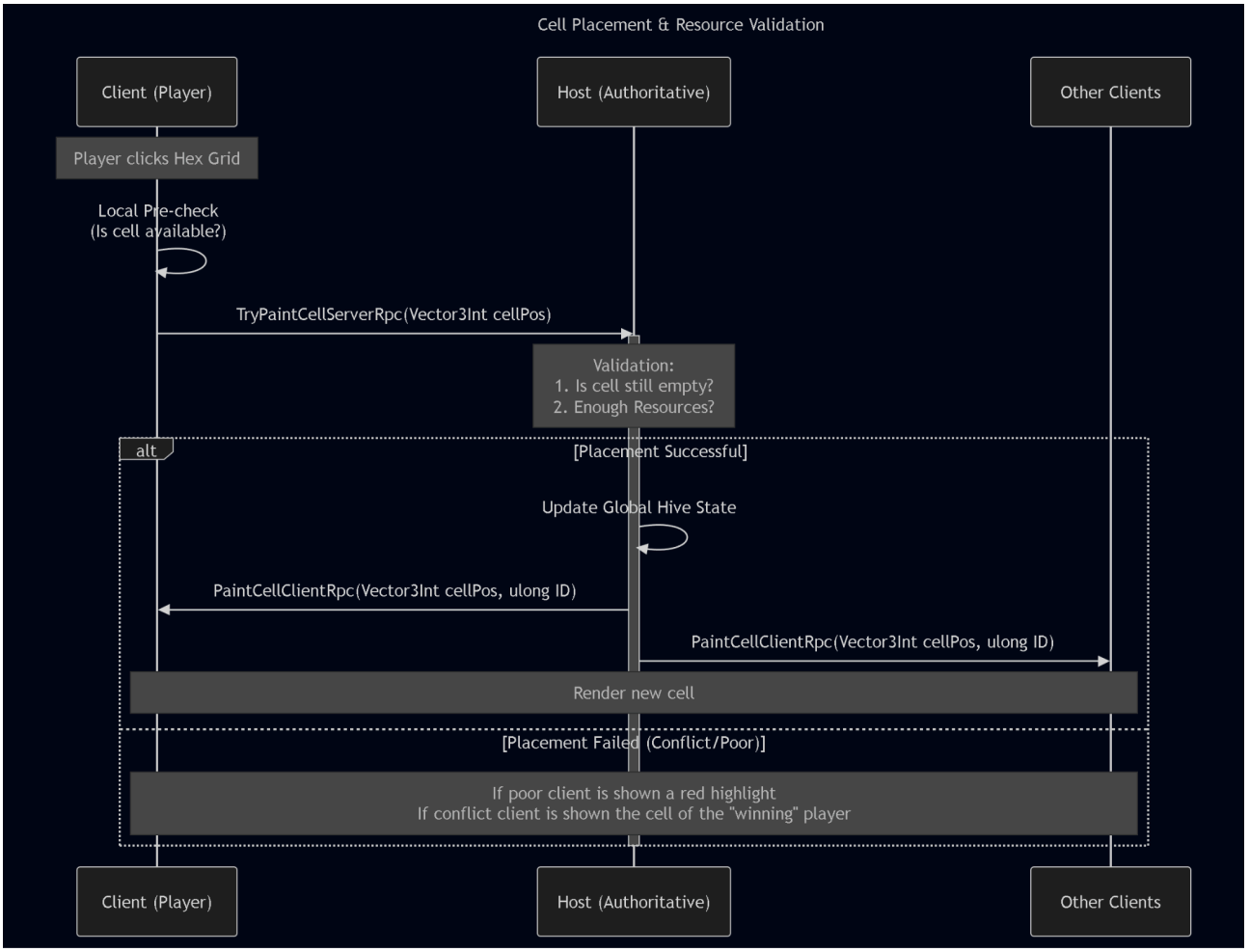
1. **Enter edit mode**: A client presses E or the specified button to enter edit mode and unless already selected, he selects the cell edit mode.

2. **Select building target**: The client uses the scroll wheel to switch to the desired building target.

3. **Click on a cell**: The client clicks on a cell with the left mouse button and sends a ServerRPC.

4. **Host evaluation**: The host receives the RPC invocation and checks if the cell can be marked as a pending addition.

5. **Feedback to client**: The host sends the result of the computation back to the builder and if the placement was a success it is also broadcasted to all other clients if they are in edit mode.


Cell Placement & Resource Validation
Client (Player)
Host (Authoritative)
Other Clients
Player clicks Hex Grid
Local Pre-check
(Is cell available?)
TryPaintCellServerRpc(Vector3Int cellPos)
Validation:
1. Is cell still empty?
2. Enough Resources?
alt
[Placement Successful]
Update Global Hive State
PaintCellClientRpc(Vector3Int cellPos, ulong ID)
PaintCellClientRpc(Vector3Int cellPos, ulong ID)
Render new cell
[Placement Failed (Conflict/Poor)]
If poor client is shown a red highlight
If conflict client is shown the cell of the "winning" player
Client (Player)
Host (Authoritative)
Other Clients

9 Delivery

9.1 Estimated Delivery Time

Prototype Phase (Completed): Focused on core mechanics, listen server infrastructure, and network data synchronization testing. More info in the game design document section 9.

Alpha Phase (Q3 2026 – Q1 2027): Core feature implementation including the three grid-based hive editing sub-modes, shared entity authority implementation, and internal stress-testing of up to 4-player sessions.

Pre-Beta & Integration (Q2 2027 – Q3 2027): Integrating the Steamworks API, setting up Unity Relay logic, configuring Unity Cloud Save, and implementing diagnostic telemetry reporting.

Open Beta Phase via Steam Playtest (Q4 2027 – Q2 2028): A 6-month public testing phase to gather performance telemetry via Unity Cloud Diagnostics and optimize server update ticks for lower-end target client devices.

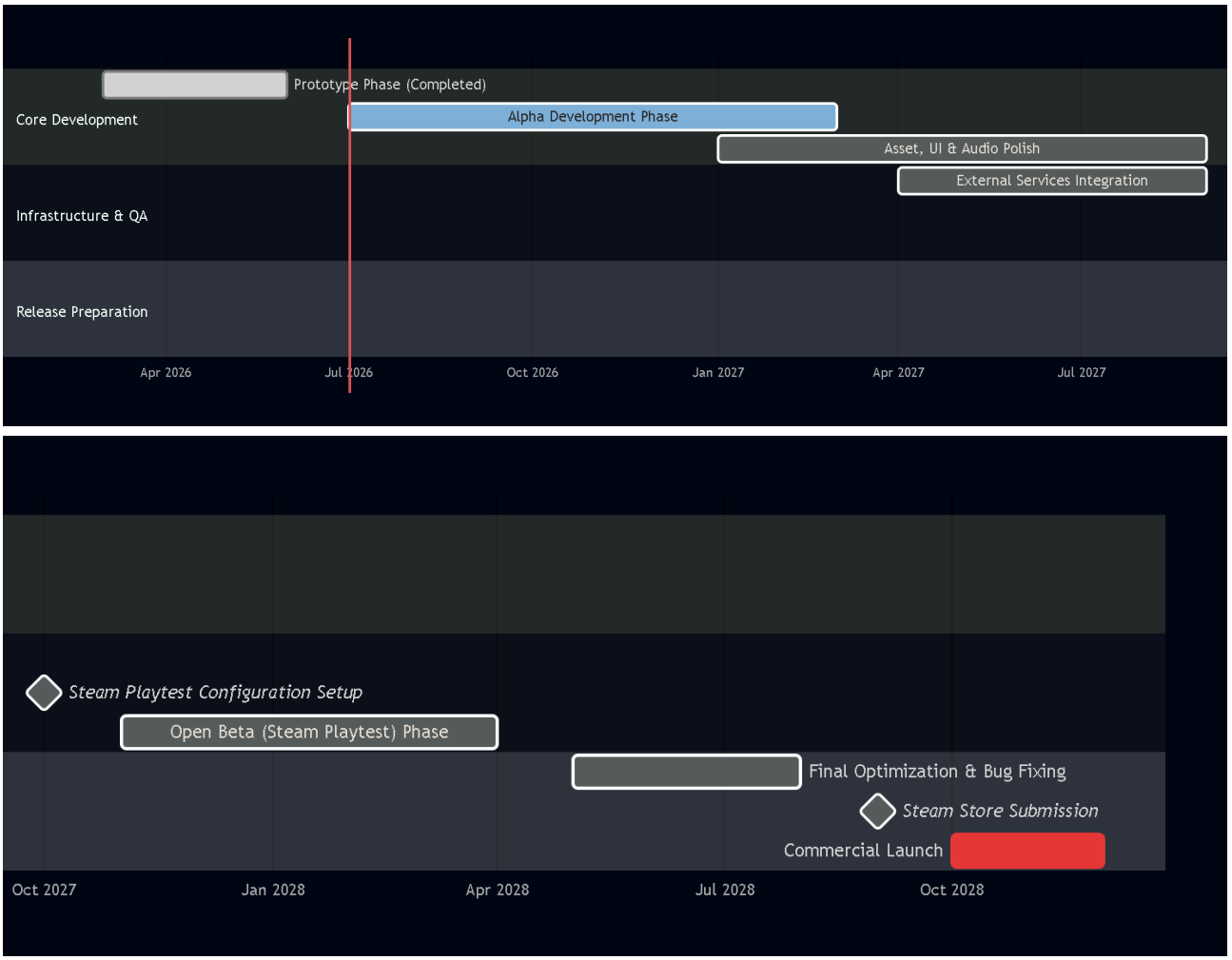
Polishing & Certification (Q3 2028): Bug fixing based on beta feedback, performance tuning for the 720p/30fps target spec, and final preparation for Steam store submission.

Full Release (Q4 2028): Official global commercial launch on Steam.

9.2 Delivery Platform

We plan on using Steam Playtest, which is a dedicated tool for beta management. This will allow us to manage more efficiently the progress of the beta and gather additional data apart from the feedback sent directly from playtesters.
All updates and patches will be delivered via Valve's Content Delivery Network. This allows us to simplify the process of providing fast updates in case of game breaking bugs and forces every playtester to play with the latest version, preventing version mismatches.

9.3 Gantt Chart



10 Cost Estimation

10.1 Physical and Operational

Source	Estimated Cost	Notes
Office rental	1200€ monthly	Estimation based on 3 people meaning 400€ monthly per person
Utilities (Electricity, Internet)	210€ monthly	Estimation based on 3 people meaning 70€ monthly per person
Cleaning services	150€ monthly	
Accounting and Legal fees	250€ monthly	
Hardware replacement and IT support	50€ monthly	

10.2 Hardware and Workplace Setup

Source	Estimated cost	Notes
Office furniture	900€	Estimated chairs, whiteboards, storage for 3 people meaning 300€ per person
Testing hardware	600€	Low end devices to match the minimum requirements for testing the game

10.3 Services

Service Provider	Estimated Cost	Notes
Unity Relay (CCU)	0.16€ per avg. CCU (past 50 CCU) Estimated: 8€ monthly	Expecting an average of 100 CCU per month
Unity Relay (Data)	0.09€ per GiB US + EU, 0.16€ per GiB Asia + Australia (past 150 GiB) Estimated: 13.50€ monthly	Expecting around 300 GiB per month
Unity Cloud Save	Free	Used for persistent JSON user profiles
Unity Cloud Diagnostic	Free	Used for collecting telemetry data
Steam Direct	100.00€	Required for submitting the

		game to the Steam store.
Steam CDN	Free	Required during the open beta for providing updates.
GitHub	Free	Essential for progress tracking and version control
Discord	Free	Provides the primary community hub and developer support channel
Netcode for GameObjects	Free	Open-source Unity multiplayer framework

10.4 Personnel

Component	**Per person/year**	**Notes**
RAL	22,000€	1,450–1,500€ net monthly
INPS employer contributions	6,490€	
INAIL	110€	Low-Risk classification
TFR accrual	1,520€ per person yearly	Mandatory severance fund
Total Employer cost per person	~30,000€/year	2,500€/month
Team of 3 × 30 months	~ 225,000€	

10.5 Company Setup

Source	**Estimated cost**	**Notes**
SRL incorporation	2,500€	One-time
PEC, digital signature	500€	One-time
Annual filings	2,500€	1,000€/year, over 30 months
Total	5,500€	

10.6 Marketing and Localization

Source	**Estimated cost**
Announcement & Launch Trailer	6,000€
Festival & Showcase	3,500€

PR Campaign	5,000€
Press kit	1,500€
Localization	6,000€
Total	22,000€

10.7 Total Budget Request

Category	**Estimated cost**
Personnel	225,000€
Physical and Operational	~55,000€
Hardware and Workplace setup	1,500€
One-time services	100€
Company Setup	5,500€
Marketing and Localization	22,000€
Subtotal	~310,000€
Contingency (~10%)	31,000€
Total budgeted need	~340,000€
Funding Request	350,000€

10.8 Monthly Burn Rate

Monthly expenses	Estimated Cost
Personnel (3 x 2,500€)	7,500€
Physical and Operational	1,860€
Services	21.50€
Total	~9**,400€ monthly**

10.9 Additional Expenses per New Hire

Add expenses per new hire	**Estimated Cost**
Physical and Operational	470€ monthly
Personnel	2,500€ monthly

Total	~3,000€ monthly

11 References

1. Listen server architecture:
   https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.4/manual/learn/listenserverhostarchitecture.html

2. Unity Relay and NAT punch-through:
   https://discussions.unity.com/t/relay-server-performance-nat-punch-through/649216

3. Unity Relay Manual:
   https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.4/manual/relay/relay.html

4. Unity Relay:
   https://docs.unity.com/en-us/relay/relay-servers

5. Unity Gaming Services pricing:
   https://unity.com/products/gaming-services/pricing

6. Steam lobbies:
   https://partner.steamgames.com/doc/features/multiplayer/matchmaking

7. Unity relay connection flow:
   https://docs.unity.com/en-us/relay/allocating-binding-joining

8. Unity relay QoS:
   https://docs.unity.com/en-us/relay/qos

9. Unity transport:
   https://docs.unity3d.com/Packages/com.unity.transport@6.5/manual/index.html

10. Unity transport pipelines:
    https://docs.unity3d.com/Packages/com.unity.transport@2.5/manual/pipelines-usage.html

11. GitHub:
    https://github.com

12. Discord:
    https://discord.com

13. Steam Direct:
    https://partner.steamgames.com/doc/gettingstarted/appfee

14. SteamDB info:
    https://steamdb.info